

Recuperatorio OO1 29/11/2025

Ejercicio 1

Se desea desarrollar una plataforma dedicada a la compra y venta de videos musicales. Las personas podrán publicar sus propios vídeos y adquirir aquellos compartidos por otros miembros de la comunidad.

De cada persona se conoce su nombre, una dirección de correo electrónico que la identifica y un saldo de créditos. Además, puede tener videos musicales puestos a la venta. Cada vez que otra persona compra uno de sus videos, recibe un ingreso que se suma automáticamente a su saldo de créditos, el cual luego puede utilizar para realizar sus propias compras dentro de la plataforma.

Los videos disponibles en la plataforma se describen mediante un título, una breve descripción, comentarios y un precio expresado en créditos. Cada comentario incluye un texto, la persona que lo escribió y la fecha en que fue realizado. Cada vez que una persona compra un vídeo, se descuentan créditos de su saldo, estos se transfieren al autor del contenido, y el vídeo adquirido se registra en su historial junto con la fecha de compra.

La plataforma debe permitir las siguientes funcionalidades:

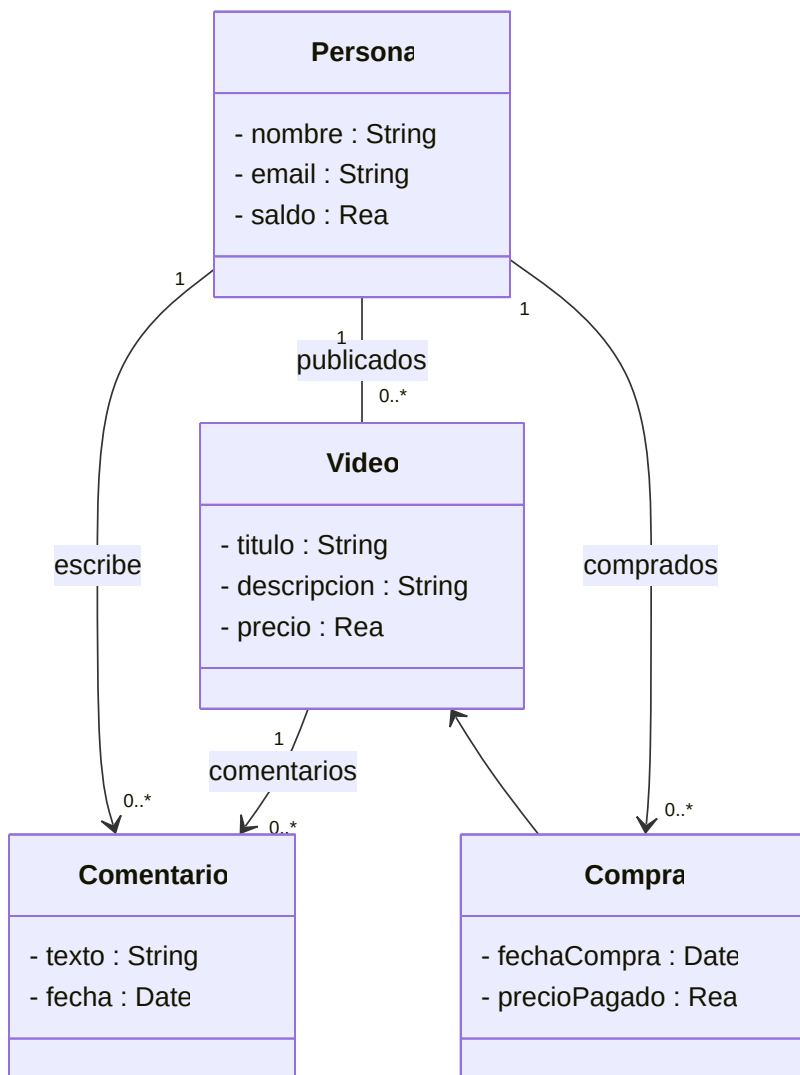
- **Recargar créditos:** una persona puede agregar créditos a su cuenta, actualizando así su saldo disponible.
- **Comprar un vídeo:** una persona puede adquirir acceso a un video siempre que cuente con créditos suficientes. Si la compra se concreta, se descuentan los creditos correspondientes de su saldo, se acreditan al autor del vídeo y el vídeo se incorpora a su historial, junto con la fecha de compra y el precio pagado en créditos. Si la persona no tiene créditos suficientes, la compra no se realiza.
- **Calcular los créditos gastados desde una fecha dada:** se debe retornar la cantidad de créditos gastados en vídeos que se hayan adquirido a partir de la fecha indicada, sumando lo gastado en cada uno
- **Cambiar el precio de un video:** El autor de un vídeo puede modificar su precio expresado en créditos. Este cambio no debe afectar el valor registrado en ventas ya realizadas.
- **Agregar comentario:** una persona puede agregar un comentario con texto a cualquier vídeo, registrándose automáticamente quién lo escribió y la fecha en que fue creado.

Tarea:

Realice un diagrama de clases UML donde identifique las clases del dominio, atributos, y las relaciones con sus roles y cardinalidades según corresponda. No debe incluir comportamiento.

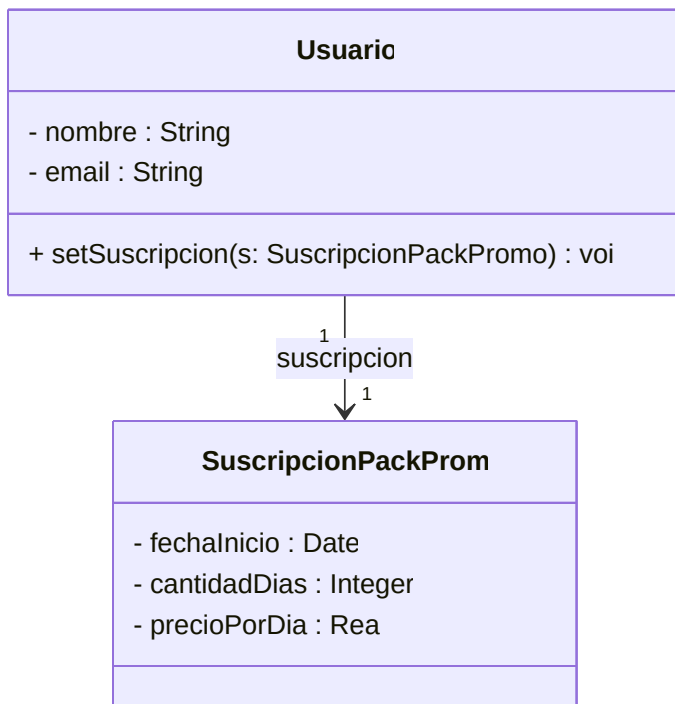
NOTA: Asegúrese que su modelo permite resolver todas las funcionalidades pedidas.

UML



Ejercicio 2

Se tiene un sistema que permite a los usuarios contratar suscripciones para acceder a servicios digitales. Actualmente en este sistema existe la **Suscripción pack promo**, que cuenta con una fecha de inicio, una duración en días y un precio por día. El sistema permite que el pago se realice incluso después de la fecha de inicio, aplicando en ese caso un **recargo**. EL monto a pagar se calcula en una fecha determinada, tomando como precio base el precio por día multiplicando por los días contratados, aplicando un recargo del 30% si el pago se realiza después de la fecha de inicio y sumando el IVA del 21%



```

public class Usuario {
    private String nombre;
    private String email;
    private SuscripcionPackPromo suscripcion;

    public Usuario(String nombre, String email,
        SuscripcionPackPromo suscripcion) {
        this.nombre = nombre;
        this.email = email;
        this.suscripcion = suscripcion;
    }

    public double montoAPagar(LocalDate fechaPago) {
        return this.suscripcion
            .montoAPagar(fechaPago);
    }
}

////////////////////////////////////

public class SuscripcionPackPromo {
    private LocalDate fechaInicio;
    private int cantidadDias;
    private double precioPorDia;

    public SuscripcionPackPromo(LocalDate fechaInicio,
        int cantidadDias, double precioPorDia){
        ...
    }

    public double calcularMontoConIVA(double valor) {
        double iva = valor * 0.21;
        return valor + iva;
    }

    public double montoAPagar (LocalDate fechaPago) {
        double costo = this.precioPorDia * this.cantidadDias;
  
```

```

    double recargo = this.calcularRecargo(fechaPago, costo);
    return this.calcularMontoConIva(costo+recargo);
}
}

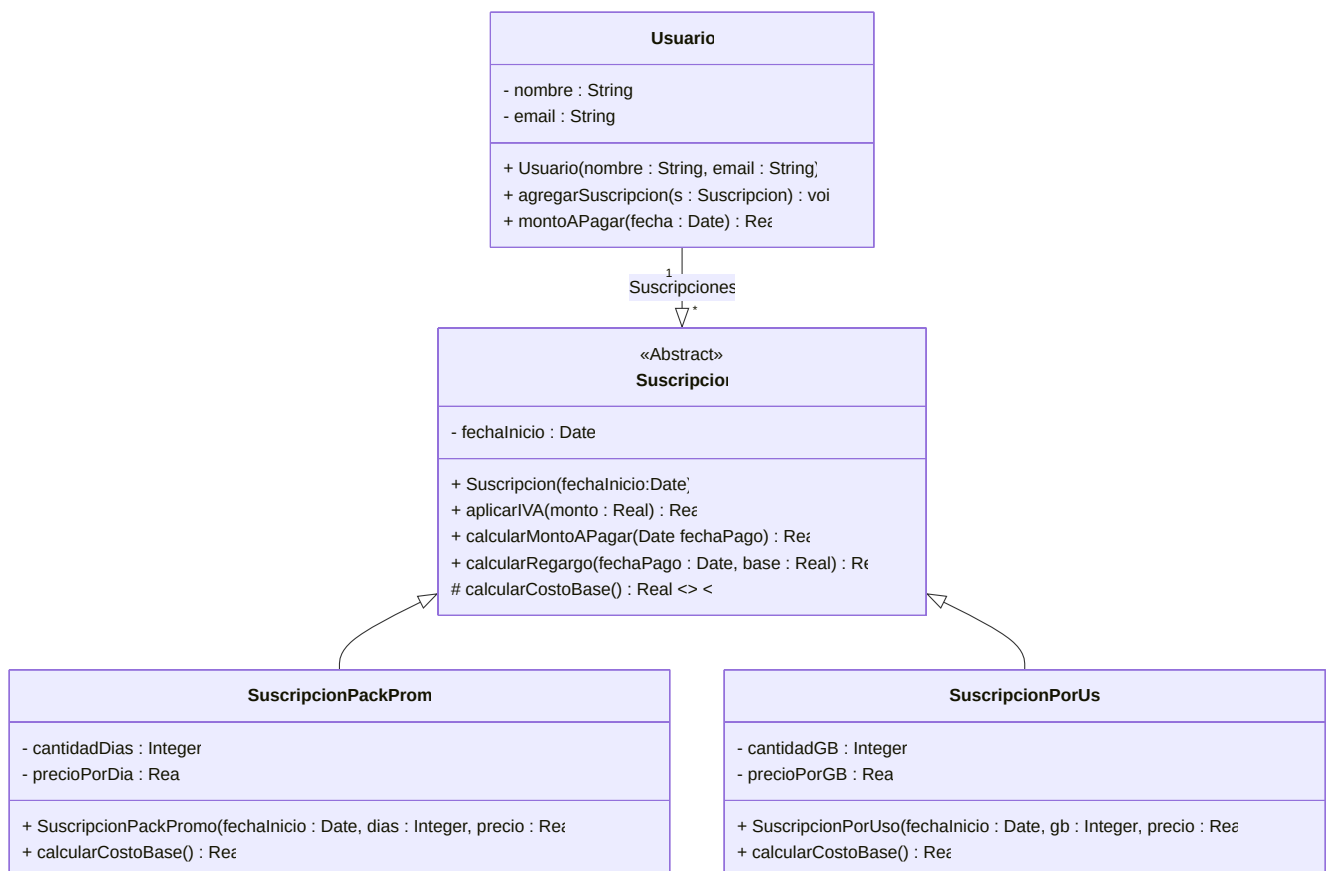
```

Se desea agregar un nuevo tipo de **suscripción por uso**, que consiste en un pack con un límite fijo de datos en GB, por ejemplo 8.4GB. Esta suscripción cuenta con una fecha de inicio, una cantidad de GB como límite y un precio por cada GB del paquete. El **monto a pagar se puede calcular en una fecha determinada**, tomando como precio base la cantidad de GB contratada multiplicada por el precio por GB, aplicando un recargo del 30% si el pago se realiza después de la fecha de inicio sumando el IVA del 21%
Además, se requiere que un usuario pueda tener **múltiples suscripciones** activas simultáneamente. El monto a pagar de un usuario es la sumatoria de todas las suscripciones activas que tenga.

Tarea:

1. Rediseñe lo necesario para permitir que la clase Usuario permita tener una colección de suscripciones, que pueden ser por uso o PackPromo, evitando duplicación de código y calculando los costos polimórficamente. Aplique los conceptos vistos en la materia. Realice un diagrama de clases UML completo de su diseño (incluyendo los métodos y constructores).
2. Reimplemente en Java respetando el nuevo diseño

Diseño UML



Implementación

usuario

```

public class Usuario {
    private String nombre;
    private String email;
    private List<Suscripcion> suscripciones;

    public Usuario (String nombre, String email) {
        this.nombre = nombre;
        this.email = email;
        this.suscripciones = new ArrayList<>();
    }

    public void agregarSuscripcion (Suscripcion suscripcion){
        this.suscripciones.add(suscripcion);
    }

    public double montoAPagar (LocalDate fechaPago) {
        return this.suscripciones.stream()
            .mapToDouble(s -> s.montoAPagar(fechaPago))
            .sum();
    }
}

```

Suscripcion

```

public abstract class Suscripcion {
    private LocalDate fechaInicio;

    public Suscripcion(LocalDate fechaInicio){
        this.fechaInicio = fechaInicio;
    }

    public double aplicarIVA(double monto){
        return monto + 1.21;
    }

    public double calcularRecargo (LocalDate fechaPago, double base){
        if (fechaPago.isAfter(this.fechaInicio)) {
            return base * 0.30;
        }
        return 0;
    }

    public double montoAPagar (LocalDate fechaPago, double base){
        double base = this.calcularCostoBase();
        double recargo = this.calcularRecargo(fechaPago, base);
        return this.aplicarIVA(base + recargo);
    }

    abstract double calcularCostoBase();
}

```

Suscripcion Pack Promo

```

public class SuscripcionPackPromo extends Suscripcion {
    private int cantidadDias;
}

```

```

private double precioPorDia:

public SuscripcionPackPromo (LocalDate fechaInicio, int dias, double precio ){
    super(fechaInicio);
    this.cantidadDias = dias;
    this.precioPorDia = precio;
}

public double calcularCostoBase (){
    return this.cantidadDias * precioPorDia;
}
}

```

Suscripcion Por Uso

```

public class SuscripcionPorUso extends Suscripcion {
    private int cantidadGB;
    private double precioPorGB;

    public SuscripcionPorUso (fechaInicio LocalDate, gb int, precio double){
        super(fechaInicio);
        this.cantidadGB = gb;
        this.precioPorGB = precio;
    }

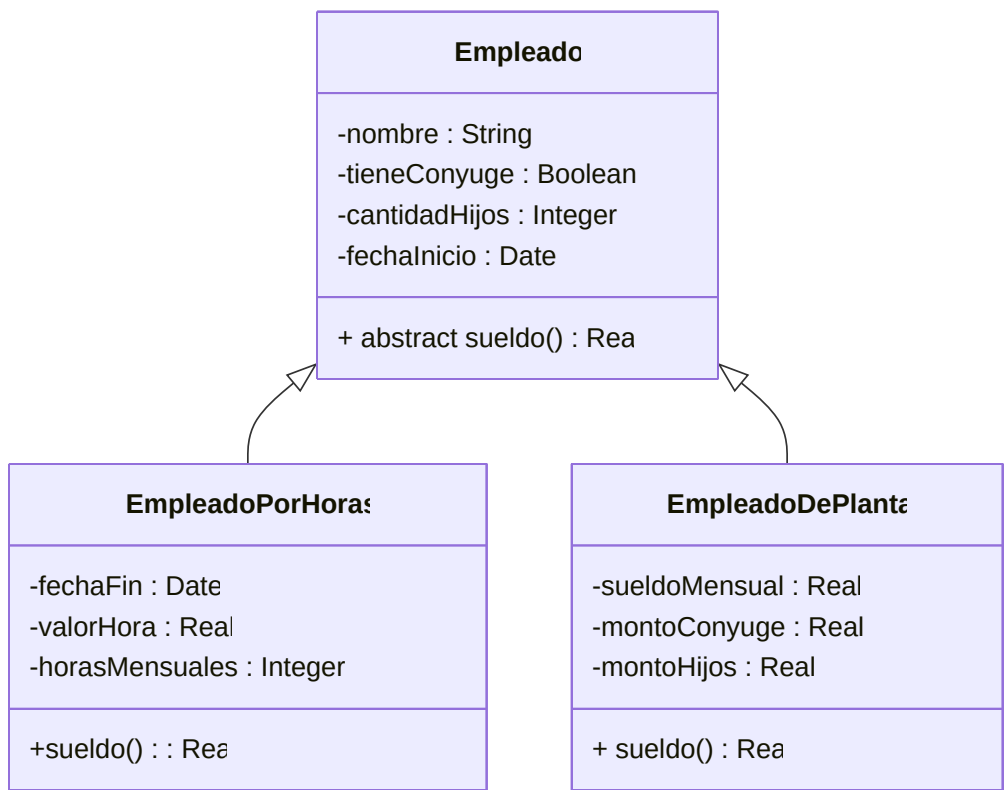
    public double calcularCostoBase(){
        return this.cantidadGB * this.precioPorGB;
    }
}

```

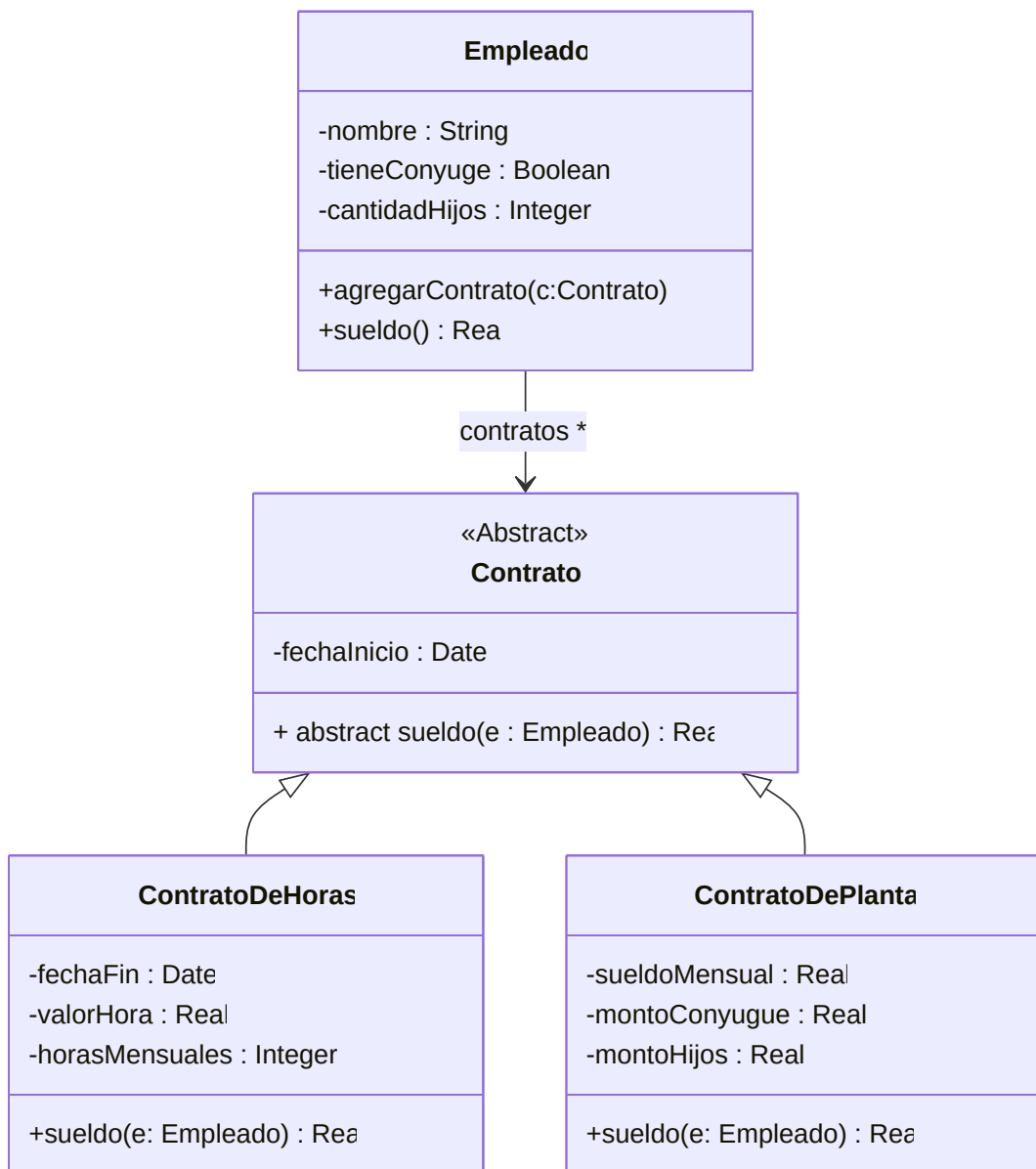
Ejercicio 3

A continuación se presentan dos posibles modelos para un sistema de administración de personal.

Solución A



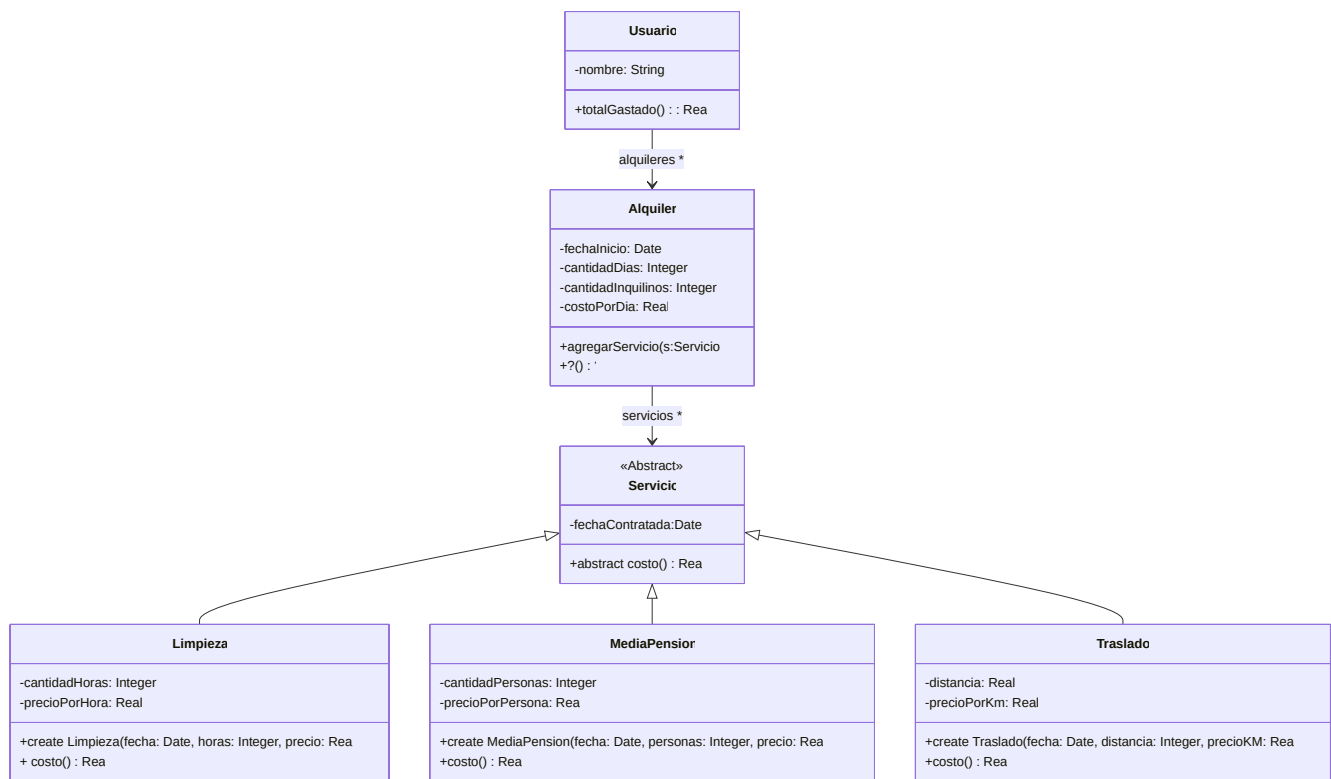
Solución B



1. Permite al empleado cambiar de contrato dinámicamente. **SOLO B**
2. Permite que un empleado tenga múltiples contratos simultáneamente. **SOLO B**
3. Es Posible extender el diseño para tener nuevos empleados con un nuevo tipo de contrato. **AMBOS**
4. Permite reutilizar la implementación de "contrato por horas" en otro sistema sin llevarme todo el código de Empleado. **SOLO B**
5. Si agrego un atributo nuevo a Empleado, automáticamente lo tienen todos los tipos de contrato. **SOLO A**
6. Permite tener un empleado sin contrato. **SOLO A**
7. Permite testear que el cálculo de sueldo funciona bien sin necesidad de crear empleados. **SOLO B**
8. El empleado delega a otro objeto el cálculo del sueldo. **SOLO B**
9. Si agrego la implementación de un nuevo tipo de contrato, la clase Empleado existente NO cambia. **SOLO B**
10. Para cambiar un contrato se debe instanciar un nuevo empleado. **Ninguno**

Ejercicio 4

Una empresa administra un sistema de alquileres que han sido contratados por distintos usuarios, cuyo diseño se representa en el siguiente diagrama de clases UML



Cada usuario puede tener múltiples alquileres, y cada alquiler puede incluir diversos servicios adicionales. El costo de un alquiler se calcula como la cantidad de días multiplicado por el costo del día, y ese resultado se le suma el costo de los servicios asociados.

Tareas:

1. Implementar el constructor de las clases Usuario, Alquiler, Servicio y Traslado

```

public Usuario (String nombre){
    this.nombre = nombre;
    this.alquileres = new ArrayList<>();
}

public Alquiler (LocalDate fechaInicio, int cantidadDias,
    int cantidadInquilinos, double costoPorDia){
    this.fechaInicio = fechaInicio;
    this.cantidadDias = cantidadDias;
    this.cantidadInquilinos= cantidadInquilinos;
    this.costoPorDia = costoPorDia;

    this.servicios = new ArrayList<>();
}

public Servicio (LocalDate fechaContratada){
    this.fechaContratada = fechaContratada;
}

public Traslado (LocalDate fecha, double distancia, double precioKm){
    super(fecha);
    this.distancia = distancia;
    this.precioKm = precioKm;
}
  
```

2. Implemente el método `totalGastado()` de la clase `Usuario`, junto con los métodos auxiliares que dicho cálculo requiera. El total gastado se define como la suma de los costos de alquileres que realizó. Utilice Streams cuando resulte apropiado. Considere que los métodos `costo de Limpieza`, `MediaPension` y `Traslado` ya están implementados.

```
// metodos en ALQUILER
public double gastadoEnAlquiler (){
    double base = cantidadDias * costoPorDia;
    return base + this.gastadoEnServicios();
}

public double gastadoEnServicios (){
    return servicios.stream()
        .mapToDouble(s -> s.costo())
        .sum();
}

// metodo en USUARIO

public double totalGastado(){
    return alquileres.stream()
        .mapToDouble(a -> a.gastadoEnAlquiler())
        .sum();
}
```

3. Instancie el siguiente usuario con sus alquileres y servicios asociados, utilizando la siguiente información:

- Usuario: Lautaro Gonzales, con 2 alquileres:
 - Alquiler 1: fecha de inicio 01/01/2025, 5 días, 2 inquilinos, costo por día 5000
 - Servicios: Limpieza para el 02/01/2025, 3 horas, 2000 la hora
 - Media Pension para 01/01/2025, 2 personas, 2500 por persona
 - Alquiler 2: fecha inicio 15/02/2025, 3 días, 1 inquilino, costo por día 6000
 - Servicio: Traslado para el 15/02/2025, 10km, precio por km 500.

Ayuda: Para instanciar un objeto `LocalDate`, pued usar `LocalDate.of(year, month, day)`

```
// El primer alquiler

Servicio s1 = new Limpieza(LocalDate.of(2025,01,02), 3, 2000);
Servicio s2 = new MediaPension(LocalDate.of(2025,01,01), 2, 2500);

Alquiler a1 = new Alquiler(LocalDate.of(2025,01,01), 5, 2, 5000);

a1.agregarServicio(s1);
a1.agregarServicio(s2);

// El segundo alquiler
Servicio s3 = new Traslado(LocalDate.of(2025,02,15), 10, 500);

Alquiler a2 = new Alquiler(LocalDate.of(2025,02,15), 3, 1, 6000);

a2.agregarServicio(s3);

// ahora la persona
```

```
Usuario u = new Usuario("Lautaro Gonzalez");
```

```
u.agregarAlquiler(a1);
```

```
u.agregarAlquiler(a2);
```